

A.5 Analysis example for Adaptive Cruise Control

The following sections present a sample to conduct our proposed analysis and use an example of Adaptive Cruise Control (ACC) of automobiles. ACC is an advanced driver-assistance system for road vehicles that automatically adjusts the vehicle speed. The control is based on several sensor information from onboard sensors, and a radar that detects an obstacle is approaching and sends feedback to the controller to brake the vehicle. The ACC must keep a safe distance from objects in front and not slow down the vehicle abruptly, to avoid a collision. The ACC must have a smooth brake and acceleration and provide a safe trip to the passengers. This CPS is complex and uses networked devices to automate human tasks and control the car components.

A.5.1 Identify business resource needs

The example of the business requirement for the step *Identify business resource needs* is shown in Table A.1.

TABLE A.1 – Example of of the step derive performance requirements of business requirements for a car brake system.

Business analysis	Requirement (RQ-01): The car must have an application to control the energy recovery system to recover kinetic energy from the brakes and store it in a battery.
Derive performance requirements from business requirements	Performance requirements: The car should recover braking energy 200ms after the brake is pressed and stop immediately after the brake is released.
Identify the business resource needs	Resources: <i>Application:</i> The car must have an application to control the energy recovery system. <i>Memory:</i> The system must have 10 MB of RAM memory reserved to the application. <i>Operating system:</i> The operating system must support the use of applications that require 10 MB of RAM memory and have memory protection and supports Twofish 256-bit encryption. <i>Processor:</i> The processor must be ARM type and have 2 GHz of clock.

A.5.2 Identify safety resource needs

The example of the safety requirement for the step *Identify safety resource needs* is shown in Table A.2.

TABLE A.2 – Example of the step to derive the performance requirements of safety mechanisms for a car brake system.

UCA type	Provided too late.
Loss scenario	UCA: Braking is commanded too late when the distance of the vehicle is too close. Causal factor: Delayed operation of the algorithm to process the radar information and instrument cluster (set of instrumentation that includes the speedometer). Recommendation (RQ-02): The car algorithm must be able to compute the radar, and instrument cluster information to provide safe control of
Refine recommendation into requirements and identify the mechanism	Recommendation (RQ-02): The car algorithm must have computational power to process the radar feedback and instrument cluster information and provide safe control to the vehicle. Mechanism: The system must have an RTOS (Real-Time Operating System) equal to RT-Linux. Use of processor with RISK architecture.
Derive performance requirements from safety requirements	Performance requirement: The car algorithm must be able to compute the radar and instrument cluster information before 10ms.
Identify the safety resource needs	Resources: <i>Operating system:</i> The operating system resource is RT-Linux and does not support memory protection and dynamic memory allocation. <i>Processor:</i> The processor must have 1.0 GHz of clock and RISK architecture.

A.5.3 Identify security resource needs

The example of the security requirement for the step *Identify security resource needs* is shown in Table A.3.

TABLE A.3 – Example of the step to derive the performance requirement of security mechanism for a car brake system.

Priority	High
Security analysis	Description: Data flowing across Car controller may be sniffed by an attacker. Depending on what type of data an attacker can read, it may be used to attack other parts of the system or simply be a disclosure of information leading to compliance violations. Requirement (RQ-03): The car operating system must have data encryption for each control action.
Identify the security mechanisms	Mechanism: The encryption must be AES-256. The processor must have 1.2 GHz of clock.
Derive performance requirements from security requirements	Performance requirement: The system must be able to encrypt (controller) and decrypt (actuator) the information at runtime.
Identify the security resource needs	Resources: <i>Operating system:</i> The operating system must support AES-256. <i>Processor:</i> The processor must have 1.2 GHz of clock.

A.5.4 Identify performance resource needs

To *Identify the need for each resource*, the analyst selects a resource to be analyzed (e.g., *Operating system*) and verifies the requirements and mechanisms for safety, security, and business. Table A.4 describes the step *Identify conflicts and reinforcements and specify the performance resource needs* for the resource *Operating system*.

The *Identification of conflicts and reinforcements between mechanisms* starts by comparing all the performance requirements that specify the resource (in this example we use the *Operating system*). In the comparison, the analyst must identify the requirements that are not compatible or are impractical to be met together. We use the concepts from Section 5.1 (*Resource Utilization Conflict*, *Mechanism Specification Conflict*, and *Reinforcement*). Table A.5 shows the identified conflicts and reinforcements.

A *Resource Utilization Conflict* occurs with requirements RQ-02 and RQ-03. The operating system of the controller may not be able to send the encrypted control action in less than 10 seconds when using the AES-256 algorithm, which compromises the satisfac-

TABLE A.4 – Example for *Identify the need for each resource* for a car brake system.

Resource	Operating system
Safety performance scenarios	UCA: Braking is commanded too late when the distance of the vehicle is too close. Causal factor: Delayed operation of the algorithm to process the radar information and instrument cluster (set of instrumentation that includes the speedometer). Requirement (RQ-02): The car algorithm must be able to compute the radar and instrument cluster information to provide safe control of the vehicle. Mechanism: The system must have an RTOS (Real-Time Operating System) equal to RT-Linux. Use of processor with RISK architecture. Safety performance requirement: The car algorithm must be able to compute the radar and instrument cluster information before 10ms. Resource: <i>Operating system:</i> The operating system resource is RT-Linux and does not support memory protection and dynamic memory allocation.
Security performance scenarios	Priority: High Description: Data flowing across Car controller may be sniffed by an attacker. Depending on what type of data an attacker can read and use it to attack other parts of the system or simply be a disclosure of information leading to compliance violations. Requirement (RQ-03): The car operating system must have data encryption for each control action. Mechanism: The encryption must be AES-256. The processor must have 1.2 GHz of clock. Security performance requirement: The system must be able to encrypt (controller) and decrypt (actuator) the information at runtime. Resource: <i>Operating system:</i> The operating system must support AES-256.
Business performance scenarios	Requirement (RQ-01): The car must have an application to control the energy recovery system to recover kinetic energy from the brakes and store it in a battery. Business performance requirement: The car should recover braking energy 200ms after the brake is pressed and stop immediately after the brake is released. Resource: <i>Operating system:</i> The operating system must support the use of applications that require 10 MB of RAM memory and have memory protection and supports Twofish 256-bit encryption.

TABLE A.5 – Example for *Identify conflicts and reinforcements between mechanisms* for a car brake system.

Requirement	Resource utilization conflict	Mechanism specification conflict	Mechanisms reinforcement
RQ-01	-	RQ-02	RQ-03
RQ-02	RQ-03	RQ-01	-
RQ-03	RQ-02	-	RQ-01

tion of RQ-02. The analyst can solve it in several ways, such as changing the encryption algorithm, changing the operating system, setting high priority in the operating system to encryption function, choosing a better processor, and others. This kind of conflict is hard to find because the mechanisms seem compatible at the first time.

A *Mechanism Specification Conflict* occurs between the requirements RQ-01 and RQ-02. The RT-Linux usually (from RQ-02) does not support memory protection and dynamic memory allocation, which is required by RQ-01. In this case, if the application of RQ-01 has some failure it can make RT-Linux halt and shut down the entire system, this is a limitation of RT-Linux. To solve this conflict the analyst may change the operating system or specify that the application must be reliable.

The *Reinforcement* occurs with requirements RQ-01 and RQ-03. RQ-01 and RQ-03 specify two distinct encryption methods with 256-bit, and both achieve the goal to protect the data with strong encrypting. In this case, the analyst must choose the mechanism that better fits the system's needs.

After identifying the conflicts and reinforcements, the analyst *Specify the resource performance needs* that solves the conflict and rationalizes the resource use. Table A.6, presents the selected performance requirement for the *Operating system* and delimits the used resource. During the *Identify conflicts and reinforcements between mechanisms* and *Specify the resource performance needs*, is expected that the analyst changes the requirements, mechanisms, or performance requirements to solve the conflicts.

In this example, is important to highlight that analyst decisions may have a great impact on the analysis process, for instance, safety analysis requires a light operating system and security analysis requires an operating system that can deal with a high number of operations efficiently. The most probable solution is to take the complex system and update the other resources' needs (e.g., processor and memory) to respect the safety and security requirements. If the analyst updates the resources needs, it will be necessary to review the other requirements that use the same updated resources, it shows the importance of the analysis of the use of resources.

This proposal suggests possible conflicts among a set of requirements in a systematic

TABLE A.6 – Example for *Select the resource performance need* for a car brake system.

	Operation system resource
Safety resource	The operating system resource is RT-Linux and does not support memory protection and dynamic memory allocation.
Security resource	The operating system must support AES-256.
Business resource	The operating system must support the use of applications that require 10 MB of RAM memory and have memory protection and supports Twofish 256-bit encryption.
Select the resource performance need	The operating system resource is VxWorks and support memory protection and dynamic memory allocation and applications that require 10 MB of RAM memory and AES-256.

way and leaves it to the analyst to solve them. The proposed approach can be performed in two ways, specifying or demanding a mechanism. The example presented in Table A.4 is performed when the analyst specifies the mechanism and its respective resource, which can lead to conflicts among the mechanisms. Furthermore, the analyst can detail the demand of resources for each mechanism, which is combined to generate the resource specification. Table A.7 presents a comparison of methods “Mechanism specification” (used as examples in previous sections) and “Mechanism demanded” (the values were estimated by the author for comparative purposes only) to conduct the analysis of the use of resources.

TABLE A.7 – Example of two ways to identify conflicts and reinforcements.

	Mechanism specification by performance requirement	Mechanism demanded by performance requirement
Business (RQ-01)	The processor must have the clock equals 2.0 GHz.	The demand of clock processor is 1.0 GHz.
Safety (RQ-02)	The processor must have the clock equals 1.0 GHz.	The demand of clock processor is 0.8 GHz.
Security (RQ-03)	The processor must have the clock equals 1.2 GHz.	The demand of clock processor is 0.4 GHz.
<i>Result</i>	<i>The processor must have the clock equals 2.0 GHz.</i>	<i>The demand of clock processor is 2.2 GHz.</i>

Using the requirements based on *Mechanism by specification* can result in analysis with mechanism conflict, and the analyst must choose the mechanism that best fits. If we use the requirements based on the *Mechanism by demand*, it can make the analysis will not have conflicts due to the resource usage but can result in impracticable requirements, such as a processor with 10.0 GHz of the clock. If the analyst specifies the mechanism in the early analysis process (e.g., in the assumptions of safety analysis), it has more potential to generate conflicts. We highlight that both types of analyses have their problems, and the analyst should make it clear if the requirement is specifying or demanding resources

(we recommend choosing one to perform the analysis), ponder to choose the performance requirements, and choose the resources that the system needs.

The proposed steps help the analyst to select a resource, analyze the requirements to find inconsistencies and conflicts (among safety, security, and business), and identify and combine resource requirements that will be used by the system.